Bici & Scoot

Sistema de arriendo de bicicletas
y scooters compartidos

Grupo 2

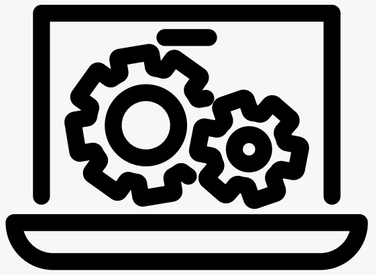
Zhi Lu
Cristobal Godoy
Nicolás Valdés
Jesús Torres
Andrés Ordoñez

Introducción y contexo del problema

El presente laboratorio consiste en el diseño e implementación de un sistema web que resuelva problematicas de gestión de datos, en nuestro caso gestionar estaciones de anclaje, vehículos (bicicletas y scooters eléctricos), usuarios, billeteras de saldo, viajes y tarifas.







Avance de proyecto

Área	Avance
Base de Datos	Tablas relacionadas, procediminetos almacenados, triggers, indices y materializada de demanda.
Backend	API REST, autenticación con JWT, permisos por rol y operaciones de viajes, billeteras y gestión administrativa.
Frontend	Login, paneles de usuario y operador, consulta de estaciones, viajes, historial, resumen mensual, tarifas y billetera.

Diagrama de arquitectura

Frontend
(React + VITE + Material UI)

Peticiones HTTP y respuesta JSON

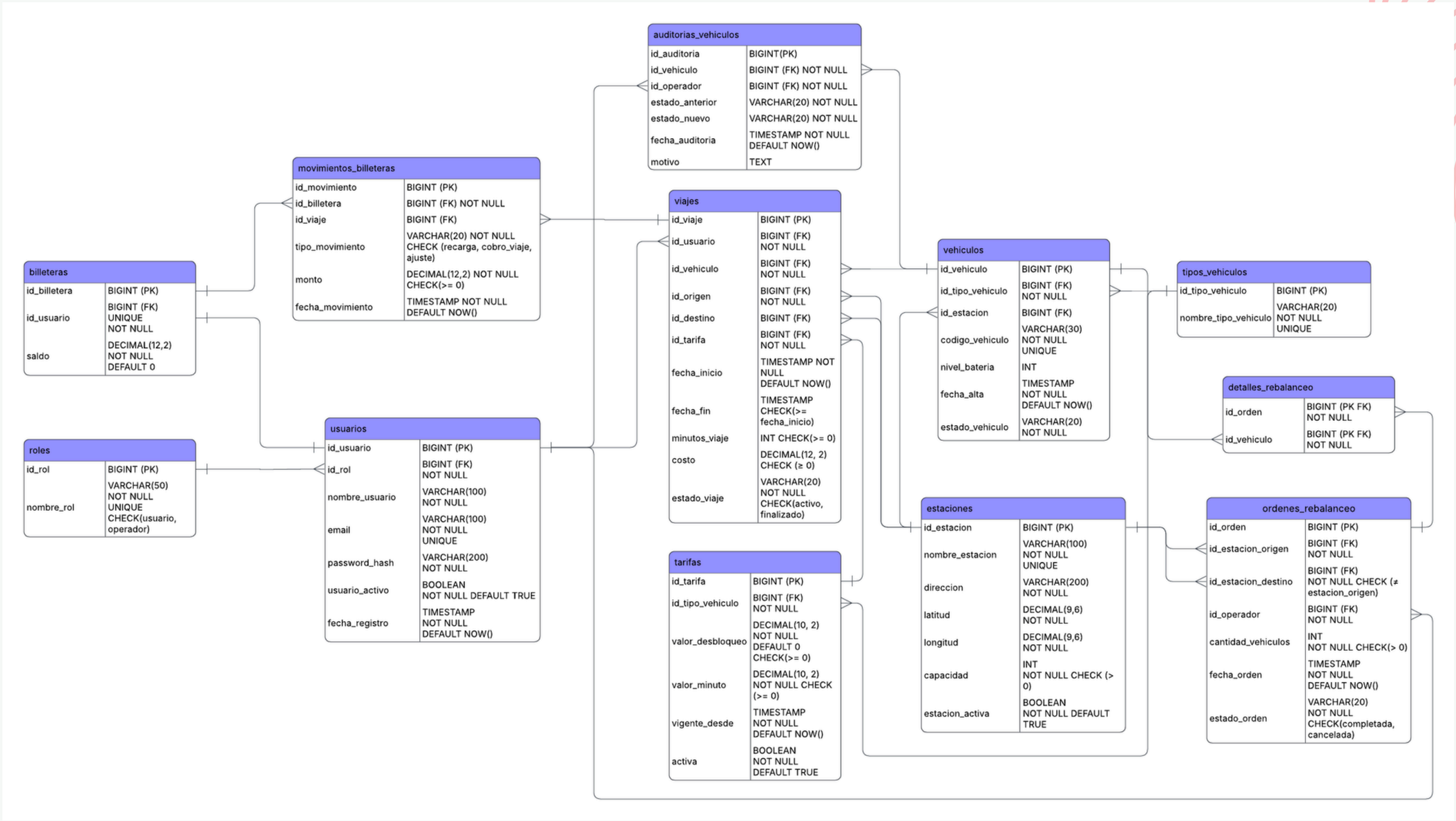
Backend
(Java 21+ Spring (Spring Boot y Security)

Consultas SQL y Resultados

Base de Datos
(PostgreSQL)

Despliegue

Diagrama MR


auditorias_vehiculos
id_auditoria BIGINT(PK)
id_vehiculo BIGINT (FK) NOT NULL
id_operador BIGINT (FK) NOT NULL
estado_anterior VARCHAR(20) NOT NULL
estado_nuevo VARCHAR(20) NOT NULL
fecha_auditoria TIMESTAMP NOT NULL DEFAULT NOW()
motivo TEXT
movimientos_billeteras
id_movimiento BIGINT (PK)
id_billetera BIGINT (FK) NOT NULL
id_viaje BIGINT (FK)
tipo_movimiento VARCHAR(20) NOT NULL CHECK (recarga, cobro_viaje, ajuste)
monto DECIMAL(12,2) NOT NULL CHECK(>= 0)
fecha_movimiento TIMESTAMP NOT NULL DEFAULT NOW()
viajes
id_viaje BIGINT (PK)
id_usuario BIGINT (FK) NOT NULL
id_vehiculo BIGINT (FK) NOT NULL
id_origen BIGINT (FK) NOT NULL
id_destino BIGINT (FK)
id_tarifa BIGINT (FK) NOT NULL
fecha_inicio TIMESTAMP NOT NULL DEFAULT NOW()
fecha_fin TIMESTAMP CHECK(>= fecha_inicio)
minutos_viaje INT CHECK(>= 0)
costo DECIMAL(12, 2) CHECK (≥ 0)
estado_viaje VARCHAR(20) NOT NULL CHECK(activo, finalizado)
billeteras
id_billetera BIGINT (PK)
id_usuario BIGINT (FK) UNIQUE NOT NULL
saldo DECIMAL(12,2) NOT NULL DEFAULT 0
roles
id_rol BIGINT (PK)
nombre_rol VARCHAR(50) NOT NULL UNIQUE CHECK(usuario, operador)
usuarios
id_usuario BIGINT (PK)
id_rol BIGINT (FK) NOT NULL
nombre_usuario VARCHAR(100) NOT NULL
email VARCHAR(100) NOT NULL UNIQUE
password_hash VARCHAR(200) NOT NULL
usuario_activo BOOLEAN NOT NULL DEFAULT TRUE
fecha_registro TIMESTAMP NOT NULL DEFAULT NOW()
vehiculos
id_vehiculo BIGINT (PK)
id_tipo_vehiculo BIGINT (FK) NOT NULL
id_estacion BIGINT (FK)
codigo_vehiculo VARCHAR(30) NOT NULL UNIQUE
nivel_bateria INT
fecha_alta TIMESTAMP NOT NULL DEFAULT NOW()
estado_vehiculo VARCHAR(20) NOT NULL
tipos_vehiculos
id_tipo_vehiculo BIGINT (PK)
nombre_tipo_vehiculo VARCHAR(20) NOT NULL UNIQUE
detalles_rebalanceo
id_orden BIGINT (PK FK) NOT NULL
id_vehiculo BIGINT (PK FK) NOT NULL
tarifas
id_tarifa BIGINT (PK)
id_tipo_vehiculo BIGINT (FK) NOT NULL
valor_desbloqueo DECIMAL(10, 2) NOT NULL DEFAULT 0 CHECK(>= 0)
valor_minuto DECIMAL(10, 2) NOT NULL CHECK (>= 0)
vigente_desde TIMESTAMP NOT NULL DEFAULT NOW()
activa BOOLEAN NOT NULL DEFAULT TRUE
estaciones
id_estacion BIGINT (PK)
nombre_estacion VARCHAR(100) NOT NULL UNIQUE
direccion VARCHAR(200) NOT NULL
latitud DECIMAL(9,6) NOT NULL
longitud DECIMAL(9,6) NOT NULL
capacidad INT NOT NULL CHECK (> 0)
estacion_activa BOOLEAN NOT NULL DEFAULT TRUE
ordenes_rebalanceo
id_orden BIGINT (PK)
id_estacion_origen BIGINT (FK) NOT NULL
id_estacion_destino BIGINT (FK) NOT NULL CHECK (≠ estacion_origen)
id_operador BIGINT (FK) NOT NULL
cantidad_vehiculos INT NOT NULL CHECK(> 0)
fecha_orden TIMESTAMP NOT NULL DEFAULT NOW()
estado_orden VARCHAR(20) NOT NULL CHECK(completada, cancelada)

Funcionalidades Solicitadas

API - CRUD (Get vehiculo)

Repository

```
public Optional<VehiculoEntity> findById(Long idVehiculo) {  Jesus Torres
    String sql = "SELECT * FROM vehiculos WHERE id_vehiculo = ?";
    return jdbcTemplate.query(sql, VEHICULO_ROW_MAPPER, idVehiculo) List<VehiculoEntity>
            .stream() Stream<VehiculoEntity>
            .findFirst();
}
```

Service

```
public VehiculoEntity obtenerVehiculoPorId(Long idVehiculo) {  5 usages  Je
    return vehiculoRepository.findById(idVehiculo)
            .orElseThrow(() -> new NoSuchElementException(
                    "Vehículo con ID " + idVehiculo + " no encontrado."));
}
```

API - CRUD (Get vehiculo)

Controller

```
@GetMapping("/{id}")   Jesus Torres
public ResponseEntity<VehiculoEntity> obtenerPorId(@PathVariable("id") Long id) {
    return ResponseEntity.ok(vehiculoService.obtenerVehiculoPorId(id));
}
```

vehiculoService.js

```
const getAll = () => http.get('/api/vehiculos');   Dkrri

const get = (id) => http.get(`/api/vehiculos/${id}`);   Dkrri

const create = (data) => http.post('/api/vehiculos', data);

const update = (id, data) =>   Dkrri
  http.put(`/api/vehiculos/${id}`, data);

const remove = (id) => http.delete(`/api/vehiculos/${id}`);
```

API/JWT - Usuario y Operadores

```java
// Tarea 2: Reglas de acceso por rol. Los usuarios inician y finalizan sus viajes y ven su historial;
// los operadores administran la flota y las estaciones.
@Bean
public SecurityFilterChain filterChain(
        HttpSecurity http,
        JwtService jwtService,
        UsuarioRepository usuarioRepository) throws Exception {
    http
            .addFilterBefore(new JwtAuthenticationFilter(jwtService, usuarioRepository),
                    beforeFilter: UsernamePasswordAuthenticationFilter.class)
            .csrf(csrf -> csrf.disable())
            .sessionManagement(sesion -> sesion.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(e -> e.authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
            .authorizeHttpRequests(rutas -> rutas
                    .requestMatchers(...patterns: "/api/auth/login", "/api/auth/logout", "/api/auth/clave-publica", "/error").permitAll()
                    .requestMatchers(HttpMethod.POST, ...patterns: "/api/viajes",
                            "/api/viajes/*/finalizar").hasRole(role: "Usuario")
                    .requestMatchers(HttpMethod.GET, ...patterns: "/api/viajes/resumen-mensual",
                            "/api/viajes/mis-viajes").hasRole(role: "Usuario")
                    .requestMatchers(HttpMethod.GET, ...patterns: "/api/viajes/resumen-mensual",
                            "/api/viajes/mis-viajes", "/api/billeteras/mi-billetera").hasRole(role: "Usuario")
                    .requestMatchers(HttpMethod.GET, ...patterns: "/api/estaciones/**", "/api/tarifas/**",
                            "/api/tipos-vehiculos/**").authenticated()
                    .requestMatchers(...patterns: "/api/estaciones/**", "/api/tarifas/**", "/api/tipos-vehiculos/**",
                            "/api/roles/**", "/api/rebalanceos/**").hasRole(role: "Operador")
                    .requestMatchers(
                            ...patterns: "/api/usuarios/**",
                            "/api/vehiculos/**",
                            "/api/billeteras/**",
                            "/api/movimientos-billeteras/**",
                            "/api/viajes/**",
                            "/api/ordenes-rebalanceo/**",
                            "/api/detalles-rebalanceo/**",
                            "/api/auditorias-vehiculos/**"
                    ).hasRole(role: "Operador")
                    .requestMatchers(...patterns: "/api/reportes/**").hasRole(role: "Operador")
                    .anyRequest().authenticated()
            );
    return http.build();
}
```

Procedimiento Almacenado 1 - Finalizar un Viaje

```sql
CREATE OR REPLACE PROCEDURE finalizar_viaje(p_id_viaje BIGINT, p_id_estacion_destino BIGINT)


LANGUAGE plpgsql AS $$
DECLARE
    -- Declaramos variables para guardar informacion
    var_minutos_uso INT;
    var_id_usuario BIGINT;
    var_id_vehiculo BIGINT;
    var_fecha_inicio TIMESTAMP;
    var_valor_minuto DECIMAL (10,2);
    var_costo_final DECIMAL (10,2);
    var_id_billetera BIGINT;

    var_hora_inicio INT;
    var_minuto_inicio INT;
    var_hora_actual INT;
    var_minuto_actual INT;

    var_capacidad_max INT;
    var_vehiculos_actuales INT;


BEGIN

    -- Bloqueamos la estación destino de forma exclusiva para evitar concurrencia
    SELECT capacidad INTO var_capacidad_max
    FROM estaciones
    WHERE id_estacion = p_id_estacion_destino
    FOR UPDATE;

    -- Contamos cuántos vehículos hay anclados en esa estación en este instante
    SELECT COUNT(*) INTO var_vehiculos_actuales
    FROM vehiculos
    WHERE id_estacion = p_id_estacion_destino;

    -- Verificamos si queda espacio
    IF var_vehiculos_actuales >= var_capacidad_max THEN
        RAISE EXCEPTION 'Error de capacidad: La estación destino está llena ', var_capacidad_max;
    END IF;


    -- Almacenamos los datos necesarios en las variables declaradas anteriormente
    SELECT fecha_inicio, id_usuario, id_vehiculo
    INTO var_fecha_inicio, var_id_usuario, var_id_vehiculo
    FROM viajes
    -- Seleccionamos solo los viajes los cuales tienen el estado 'Activo' es decir los que aun no t
    WHERE id_viaje = p_id_viaje AND estado_viaje = 'Activo';


    IF NOT FOUND THEN
```

```sql
IF NOT FOUND THEN
    RAISE EXCEPTION 'Error: El viaje % no existe o ya está Finalizado.', p_id_viaje;
END IF;

-- Calculamos los minutos en uso del vehiculo tras su trayecto, tomamos los minutos de inicio en el cual s

-- Extraemos la hora y minuto de el atributo fecha_inicio de viajes
SELECT EXTRACT(HOUR FROM var_fecha_inicio), EXTRACT(MINUTE FROM var_fecha_inicio)
INTO var_hora_inicio, var_minuto_inicio;

-- Extraemos la hora y minuto actual
SELECT EXTRACT(HOUR FROM NOW()), EXTRACT(MINUTE FROM NOW())
INTO var_hora_actual, var_minuto_actual;

-- Si la hora actual es menor a la inical, (caso en que sea media noche) le sumamos 24 hrs (un dia)
IF var_hora_actual < var_hora_inicio THEN
    var_hora_actual := var_hora_actual + 24;
END IF;

-- Calculamos el costo según la tarifa y los minutos utilizados
var_minutos_uso := ((var_hora_actual - var_hora_inicio) * 60) + (var_minuto_actual - var_minuto_inicio);


-- Obtener el valor por minuto del tipo de vehículo utilizado
SELECT tarifas.valor_minuto
INTO var_valor_minuto
FROM vehiculos
JOIN tarifas ON vehiculos.id_tipo_vehiculo = tarifas.id_tipo_vehiculo
WHERE vehiculos.id_vehiculo = var_id_vehiculo AND tarifas.activa = TRUE;

-- Calculamos el costo total
var_costo_final := var_minutos_uso * var_valor_minuto;

-- Actualizamos la tabla de Viajes
UPDATE viajes
SET fecha_fin = NOW(),
    minutos_viaje = var_minutos_uso,
    costo = var_costo_final,
    id_destino = p_id_estacion_destino,
    estado_viaje = 'Finalizado'
WHERE id_viaje = p_id_viaje;

-- Guardamos el id de la billetera asociado al usuario correspondiente
SELECT id_billetera
INTO var_id_billetera
FROM billeteras
WHERE id_usuario = var_id_usuario;
```

Procedimiento Almacenado 1 - Finalizar un Viaje

```
    -- Descontamos el dinero de la billetera
    UPDATE billeteras
    SET saldo = saldo - var_costo_final
    WHERE id_usuario = var_id_usuario;

    -- Registrar el cobro en el historial de movimientos de la billetera
    INSERT INTO movimientos_billeteras (id_billetera, id_viaje, tipo_movimiento, monto, fecha_movimiento)
    VALUES (var_id_billetera, p_id_viaje, 'Cobro_viaje', var_costo_final, NOW());


    -- Actualizar el vehículo para anclarlo en la estación de destino
    UPDATE vehiculos
    SET id_estacion = p_id_estacion_destino,
        estado_vehiculo = 'Disponible'
    WHERE id_vehiculo = var_id_vehiculo;

END;
```

Procedimiento Almacenado 2 - Rebalanceo de flota

```sql
-- Tarea 4: Procedimiento 2. Rebalancea la flota: traslada vehículos de un tipo entre dos estaciones,
-- valida capacidad y operador, y registra la orden de traslado.

CREATE OR REPLACE PROCEDURE rebalancear_flota(
    p_id_estacion_origen BIGINT,
    p_id_estacion_destino BIGINT,
    p_id_operador BIGINT,
    p_cantidad_vehiculos INT,
    p_id_tipo_vehiculo BIGINT
)
LANGUAGE plpgsql AS $$
DECLARE
    var_capacidad_destino INT;
    var_ocupacion_destino BIGINT;
    var_vehiculos BIGINT[];
    var_id_orden BIGINT;
BEGIN
    -- No admite REPEATABLE READ: conservaría una ocupación desactualizada.
    IF current_setting('transaction_isolation') = 'repeatable read' THEN
        RAISE EXCEPTION 'rebalancear_flota no admite REPEATABLE READ; utilice READ COMMITTED o SERIALIZABLE.';
    END IF;

    IF p_cantidad_vehiculos IS NULL OR p_cantidad_vehiculos <= 0 THEN
        RAISE EXCEPTION 'La cantidad de vehículos debe ser mayor que cero.';
    END IF;

    IF p_id_estacion_origen IS NULL OR p_id_estacion_destino IS NULL THEN
        RAISE EXCEPTION 'Debe indicar las estaciones de origen y destino.';
    END IF;

    IF p_id_estacion_origen = p_id_estacion_destino THEN
        RAISE EXCEPTION 'Las estaciones de origen y destino deben ser diferentes.';
    END IF;

    -- Bloqueo de estaciones en orden ascendente para evitar interbloqueos.
    PERFORM id_estacion
    FROM estaciones
    WHERE id_estacion IN (p_id_estacion_origen, p_id_estacion_destino)
    ORDER BY id_estacion
    FOR UPDATE;

    PERFORM 1
    FROM estaciones
    WHERE id_estacion = p_id_estacion_origen AND estacion_activa = TRUE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'La estación de origen % no existe o está inactiva.', p_id_estacion_origen;
    END IF;

    SELECT capacidad INTO var_capacidad_destino
    FROM estaciones
    WHERE id_estacion = p_id_estacion_destino AND estacion_activa = TRUE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'La estación de destino % no existe o está inactiva.', p_id_estacion_destino;
    END IF;

    -- Operador vigente con rol Operador.
    PERFORM 1
    FROM usuarios u
    JOIN roles r ON r.id_rol = u.id_rol
    WHERE u.id_usuario = p_id_operador
      AND u.usuario_activo = TRUE
      AND r.nombre_rol = 'Operador'
    FOR SHARE OF u, r;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El usuario % no existe, está inactivo o no tiene el rol Operador.', p_id_operador;
    END IF;

    -- El tipo de vehículo es obligatorio y debe existir.
    IF p_id_tipo_vehiculo IS NULL THEN
        RAISE EXCEPTION 'Debe indicar el tipo de vehículo a trasladar.';
    END IF;

    PERFORM 1
    FROM tipos_vehiculos
    WHERE id_tipo_vehiculo = p_id_tipo_vehiculo;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El tipo de vehículo % no existe.', p_id_tipo_vehiculo;
    END IF;

    -- Vehículos disponibles del tipo, por id ascendente.
    SELECT ARRAY(
        SELECT id_vehiculo
        FROM vehiculos
        WHERE id_estacion = p_id_estacion_origen
          AND estado_vehiculo = 'Disponible'
          AND id_tipo_vehiculo = p_id_tipo_vehiculo
        ORDER BY id_vehiculo
        LIMIT p_cantidad_vehiculos
        FOR UPDATE
    ) INTO var_vehiculos;
```

Procedimiento Almacenado 2 - Rebalanceo de flota

```
    IF cardinality(var_vehiculos) < p_cantidad_vehiculos THEN
        RAISE EXCEPTION 'La estación de origen % tiene solo % vehículos disponibles del tipo seleccionado; se solicitaron %.',
            p_id_estacion_origen, cardinality(var_vehiculos), p_cantidad_vehiculos;
    END IF;

    -- Todo vehículo anclado al destino ocupa espacio, de cualquier estado.
    SELECT COUNT(*) INTO var_ocupacion_destino
    FROM vehiculos
    WHERE id_estacion = p_id_estacion_destino;

    IF var_ocupacion_destino + p_cantidad_vehiculos > var_capacidad_destino THEN
        RAISE EXCEPTION 'La estación de destino % no tiene capacidad para recibir % vehículos (ocupación: %, capacidad: %).',
            p_id_estacion_destino, p_cantidad_vehiculos, var_ocupacion_destino, var_capacidad_destino;
    END IF;

    -- Traslado de los vehículos seleccionados.
    UPDATE vehiculos
    SET id_estacion = p_id_estacion_destino
    WHERE id_vehiculo = ANY(var_vehiculos);

    INSERT INTO ordenes_rebalanceo (
        id_estacion_origen, id_estacion_destino, id_operador,
        cantidad_vehiculos, estado_orden
    )
    VALUES (
        p_id_estacion_origen, p_id_estacion_destino, p_id_operador,
        p_cantidad_vehiculos, 'Completada'
    )
    RETURNING id_orden INTO var_id_orden;

    INSERT INTO detalles_rebalanceo (id_orden, id_vehiculo)
    SELECT var_id_orden, unnest(var_vehiculos);

END;
$$;
```

Trigger 1 - Impedir un Viaje

```sql
-- Tarea 5: Trigger 1. Impide iniciar un viaje con saldo negativo, con otro viaje activo
-- o con un scooter bajo el 20% de bateria.

CREATE OR REPLACE FUNCTION validar_inicio_viaje()
RETURNS TRIGGER
AS
$BODY$
DECLARE
    v_saldo DECIMAL(12,2);
    v_bateria INT;
    v_tipo VARCHAR(20);
BEGIN
    -- Saldo del usuario.
    SELECT saldo INTO v_saldo
    FROM billeteras
    WHERE id_usuario = NEW.id_usuario;

    -- Bateria y tipo del vehiculo.
    SELECT v.nivel_bateria, tv.nombre_tipo_vehiculo
    INTO v_bateria, v_tipo
    FROM vehiculos v
    JOIN tipos_vehiculos tv ON v.id_tipo_vehiculo = tv.id_tipo_vehiculo
    WHERE v.id_vehiculo = NEW.id_vehiculo;

    -- Saldo negativo.
    IF v_saldo < 0 THEN
        RAISE EXCEPTION 'No puede iniciar el viaje: tiene saldo negativo ($%).', v_saldo;
    END IF;

    -- Bateria insuficiente en scooters.
    IF v_tipo = 'Scooter' AND (v_bateria IS NULL OR v_bateria < 20) THEN
        RAISE EXCEPTION 'Nivel de bateria insuficiente, no se puede iniciar el viaje.';
    END IF;

    -- Viaje activo existente.
    IF EXISTS (
        SELECT 1
        FROM viajes
        WHERE id_usuario = NEW.id_usuario AND estado_viaje = 'Activo'
    ) THEN
        RAISE EXCEPTION 'El usuario ya tiene un viaje en curso.';
    END IF;

    RETURN NEW;
END;
$BODY$
LANGUAGE plpgsql;
```

```sql
CREATE TRIGGER trigger_validar_inicio_viaje
BEFORE INSERT ON viajes
FOR EACH ROW
WHEN (NEW.estado_viaje = 'Activo')
EXECUTE FUNCTION validar_inicio_viaje();
```

Se impide el viaje si:

- Tiene saldo negativo
- Posee otro viaje activo
- La batería del vehículo es inferior al 20%.

Trigger 2 - Auditar estado

```sql
-- Tarea 6: Trigger 2. Audita el cambio de estado de un vehículo a "En mantención" o "Dado de baja"
-- con el operador responsable, la fecha y el motivo.

CREATE OR REPLACE FUNCTION auditar_cambio_estado_vehiculo()
RETURNS TRIGGER
AS
$BODY$
DECLARE
    v_id_operador BIGINT;
    v_motivo TEXT;
BEGIN
    -- Solo se audita el paso a mantención o baja.
    IF NEW.estado_vehiculo IN ('En mantención', 'Dado de baja') AND NEW.estado_vehiculo IS DISTINCT FROM OLD.estado_vehiculo
    THEN
        -- Operador y motivo enviados por el backend.
        v_id_operador := current_setting('app.id_operador')::BIGINT;
        v_motivo := current_setting('app.motivo', true);

        -- Registro de auditoría.
        INSERT INTO auditorias_vehiculos (
            id_vehiculo,
            id_operador,
            estado_anterior,
            estado_nuevo,
            fecha_auditoria,
            motivo
        )
        VALUES (
            NEW.id_vehiculo,
            v_id_operador,
            OLD.estado_vehiculo,
            NEW.estado_vehiculo,
            NOW(),
            v_motivo
        );
    END IF;

    RETURN NEW;
END;
$BODY$
LANGUAGE plpgsql;

CREATE TRIGGER trigger_auditar_cambio_estado_vehiculo
AFTER UPDATE OF estado_vehiculo ON vehiculos
FOR EACH ROW
EXECUTE FUNCTION auditar_cambio_estado_vehiculo();

-- Extra: Crea la billetera con saldo 0 de cada usuario nuevo con rol "Usuario".
CREATE OR REPLACE FUNCTION crear_billetera_usuario()
RETURNS TRIGGER
```

Audita cada cambio de estado de un vehículo a "En mantención" o "Dado de baja", guardando el operador responsable, la fecha y el motivo.

```sql
        VALUES (
            NEW.id_vehiculo,
            v_id_operador,
            OLD.estado_vehiculo,
            NEW.estado_vehiculo,
            NOW(),
            v_motivo
        );
    END IF;

    RETURN NEW;
END;
$BODY$
LANGUAGE plpgsql;

CREATE TRIGGER trigger_auditar_cambio_estado_vehiculo
AFTER UPDATE OF estado_vehiculo ON vehiculos
FOR EACH ROW
EXECUTE FUNCTION auditar_cambio_estado_vehiculo();
```

Vista Materializada

```sql
-- Tarea 7: Vista materializada de la demanda de viajes por estación y franja horaria.

CREATE MATERIALIZED VIEW mv_demanda_estacion_horaria AS
WITH viajes_iniciados AS (
    SELECT
        id_origen AS id_estacion,
        EXTRACT(HOUR FROM fecha_inicio)::INT AS franja_horaria,
        COUNT(*) AS cantidad_viajes
    FROM viajes
    GROUP BY id_origen, EXTRACT(HOUR FROM fecha_inicio)::INT
),
viajes_finalizados AS (
    SELECT
        id_destino AS id_estacion,
        EXTRACT(HOUR FROM fecha_fin)::INT AS franja_horaria,
        COUNT(*) AS cantidad_viajes
    FROM viajes
    GROUP BY id_destino, EXTRACT(HOUR FROM fecha_fin)::INT
)
SELECT
    e.id_estacion,
    e.nombre_estacion,
    h.hora AS franja_horaria,
    COALESCE(vi.cantidad_viajes, 0) AS viajes_iniciados,
    COALESCE(vf.cantidad_viajes, 0) AS viajes_finalizados
FROM estaciones e
CROSS JOIN generate_series(0, 23) AS h(hora)
LEFT JOIN viajes_iniciados   vi ON e.id_estacion = vi.id_estacion AND h.hora = vi.franja_horaria
LEFT JOIN viajes_finalizados vf ON e.id_estacion = vf.id_estacion AND h.hora = vf.franja_horaria
WITH DATA;ta
```

```sql
-- Índice único requerido por REFRESH CONCURRENTLY.
CREATE UNIQUE INDEX idx_mv_demanda_estacion_hora
ON mv_demanda_estacion_horaria (id_estacion, franja_horaria);

-- Función que refresca la vista.
CREATE OR REPLACE FUNCTION fn_refrescar_demanda_estaciones()
RETURNS void AS $$
BEGIN
    REFRESH MATERIALIZED VIEW CONCURRENTLY mv_demanda_estacion_horaria;
END;
$$ LANGUAGE plpgsql;

-- Refresco automático cada hora.
CREATE EXTENSION IF NOT EXISTS pg_cron;

SELECT cron.schedule(
    'refrescar_demanda_estaciones',
    '0 * * * *',
    $$SELECT fn_refrescar_demanda_estaciones();$$
);
```

Muestra la cantidad de viajes iniciados y finalizados por estación en cada hora del día, refrescada cada hora.

Indices - Estado/Usuario/Viaje

```sql
-- Índices y triggers

-- Tarea 8: Índice sobre el estado del vehículo.

CREATE INDEX idx_vehiculos_estado ON vehiculos(estado_vehiculo);

-- Tarea 8: Índice sobre el usuario del viaje.

CREATE INDEX idx_viajes_usuario ON viajes(id_usuario);

-- Tarea 8: Índice sobre la fecha de inicio del viaje.

CREATE INDEX idx_viajes_fecha_inicio ON viajes(fecha_inicio);
```

Endpoint - Listar estaciones

Controller

```java
@RestController  no usages  Jesus Torres +2
@RequestMapping("/api/estaciones")
public class EstacionController {

    private final EstacionService estacionService;  no usages

    public EstacionController(EstacionService estacionService) { this.estacionService = estacionService; }

    // Endpoint para obtener la disponibilidad de todas las estaciones.
    @GetMapping("/disponibilidad")  no usages  Nicolás
    public List<DisponibilidadEstacionDTO> obtenerDisponibilidad() { return estacionService.obtenerDisponibilidad(); }
```

Service

```java
// Obtener la disponibilidad de todas las estaciones.
public List<DisponibilidadEstacionDTO> obtenerDisponibilidad() {
    return estacionRepository.obtenerDisponibilidad();
}
```

Endpoint - Listar estaciones: Repository

```
public List<DisponibilidadEstacionDTO> obtenerDisponibilidad() {

    String sql = """
            SELECT
                e.id_estacion,
                e.nombre_estacion,
                COUNT(*) FILTER (
                    WHERE tv.nombre_tipo_vehiculo = 'Bicicleta'
                ) AS cantidad_bicicletas,
                COUNT(*) FILTER (
                    WHERE tv.nombre_tipo_vehiculo = 'Scooter'
                ) AS cantidad_scooters,
                ROUND(
                    AVG(v.nivel_bateria) FILTER (
                        WHERE tv.nombre_tipo_vehiculo = 'Scooter'
                    ), 2
                ) AS bateria_promedio
            FROM estaciones e
            JOIN vehiculos v
                ON e.id_estacion = v.id_estacion
            JOIN tipos_vehiculos tv
                ON v.id_tipo_vehiculo = tv.id_tipo_vehiculo
            WHERE v.estado_vehiculo = 'Disponible'
            GROUP BY e.id_estacion, e.nombre_estacion
            ORDER BY e.id_estacion
            """;
```

```
    return jdbcTemplate.query(sql, (rs, rowNum) ->
            new DisponibilidadEstacionDTO(
                    rs.getLong("id_estacion"),
                    rs.getString("nombre_estacion"),
                    rs.getLong("cantidad_bicicletas"),
                    rs.getLong("cantidad_scooters"),
                    rs.getBigDecimal("bateria_promedio")
            )
    );
}
```

Endpoint - Listar estaciones: DTO

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class DisponibilidadEstacionDTO {

    private Long idEstacion;
    private String nombreEstacion;
    private Long cantidadBicicletas;
    private Long cantidadScooters;
    private BigDecimal bateriaPromedio;
}
```

Endpoint - Resumen mensual

```java
public ResumenMensualDTO obtenerResumenMensual(Long idUsuario, int anio, int mes) {
    return jdbcTemplate.queryForObject(
            sql: "SELECT cantidad_viajes, minutos_totales, gasto_total FROM fn_resumen_mensual_usuario(?, ?, ?)",
            (rs, row) -> new ResumenMensualDTO(
                    anio,
                    mes,
                    rs.getLong(columnLabel: "cantidad_viajes"),
                    rs.getLong(columnLabel: "minutos_totales"),
                    rs.getBigDecimal(columnLabel: "gasto_total")),
            idUsuario, anio, mes);
}
```

```sql
CREATE OR REPLACE FUNCTION fn_resumen_mensual_usuario(
    p_id_usuario BIGINT,
    p_anio INT,
    p_mes INT
)
RETURNS TABLE (
    cantidad_viajes BIGINT,
    minutos_totales BIGINT,
    gasto_total NUMERIC
)
LANGUAGE sql STABLE AS $$
    SELECT
        COUNT(*),
        COALESCE(SUM(minutos_viaje), 0),
        COALESCE(SUM(costo), 0)
    FROM viajes
    WHERE id_usuario = p_id_usuario
      AND estado_viaje = 'Finalizado'
      AND fecha_inicio >= make_date(p_anio, p_mes, 1)
      AND fecha_inicio <  make_date(p_anio, p_mes, 1) + INTERVAL '1 month';
$$;
```

Endpoint que devuelva el resumen mensual de un usuario (cantidad de viajes, minutos totales y gasto) calculado directamente en la base de datos.

Demostración

Conclusiones

Bici & Scoot demuestra cómo la integración de una interfaz web, una API y una base de datos permite gestionar un servicio de movilidad compartida. El sistema conecta la disponibilidad de vehículos, los viajes y sus cobros, ofreciendo funciones específicas para usuarios y operadores.

Este proyecto nos permitió aplicar conocimientos de desarrollo web, seguridad y bases de datos, comprendiendo la importancia de mantener información consistente y de coordinar el trabajo en equipo.

Contribuciones



¡Gracias!

